



UNIVERSIDADE ESTADUAL DA PARAÍBA
BACHARELADO EM CIÊNCIAS DA COMPUTAÇÃO

ARTHUR MARQUES ARAUJO

GUILHERME SILVA HENRIQUES DE SOUSA

LUIZ JOSÉ MENDONÇA DUARTE

MANOEL BARROS DA CRUZ NETO

RELEASE 3

CAMPINA GRANDE

2026

DESCRIÇÃO DA ARQUITETURA E IMPLEMENTAÇÕES

ARQUITETURA

O sistema UberPB foi projetado utilizando uma arquitetura em camadas (Layered Architecture), o que garante uma forte separação de responsabilidades (Separation of Concerns) e facilita a expansão do domínio original de mobilidade (Uber) para o novo domínio de delivery (Uber Eats). A aplicação opera exclusivamente via interface de Linha de Comando (CLI) e é dividida nas seguintes camadas principais:

- **Camada de Apresentação (CLI - View/Controller):** Composta pelos pacotes `cli.menus` e `cli.forms`. É responsável por capturar as entradas do usuário, exibir os menus interativos e orquestrar o fluxo de navegação. Nesta release, foram introduzidos os painéis dedicados ao `MenuRestauranteCLI`, `MenuEntregadorCLI` e o fluxo complexo de montagem de carrinho na `PedidoCLI`.
- **Camada de Serviços (Business Logic):** Localizada no pacote `services`. Contém as regras de negócio isoladas da interface e da persistência. Destaca-se a implementação do `EstimativaService`, que agora processa o algoritmo probabilístico de alta demanda para o delivery (tarifa dinâmica), e as lógicas de validação estrita no `RestauranteService` e `EntregadorService`.
- **Camada de Persistência (Repository):** Implementa o padrão *Repository* (`repository.json`). Os dados são persistidos em arquivos estruturados JSON. A classe `BaseRepository` utiliza a API de concorrência do Java (`ReentrantReadWriteLock`) para garantir que múltiplas operações de leitura e escrita ocorram de forma segura (Thread-Safety). O acesso aos repositórios é centralizado através do padrão *Facade* implementado na classe `DatabaseManager`.
- **Camada de Domínio (Model):** Representa as entidades do sistema. O modelo foi expandido utilizando o princípio de Herança, onde `Restaurante` e `Entregador` estendem a classe base abstrata `User`. A nova entidade central desta release é a classe `Pedido`, que agrega instâncias de `ItemPedido` e gerencia seu próprio estado financeiro e temporal.

DIAGRAMA DE CASO DE USO

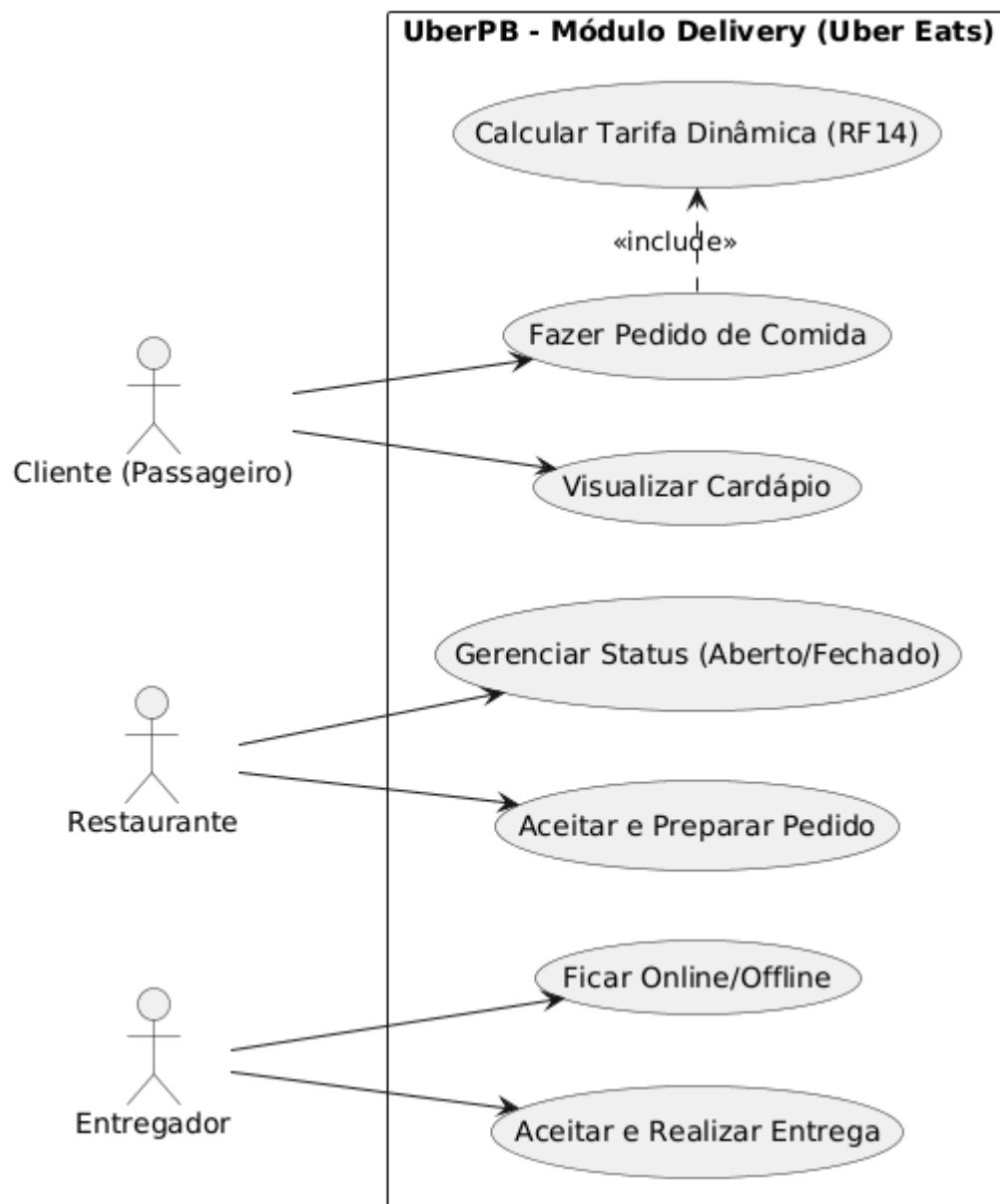


DIAGRAMA DE CLASSE

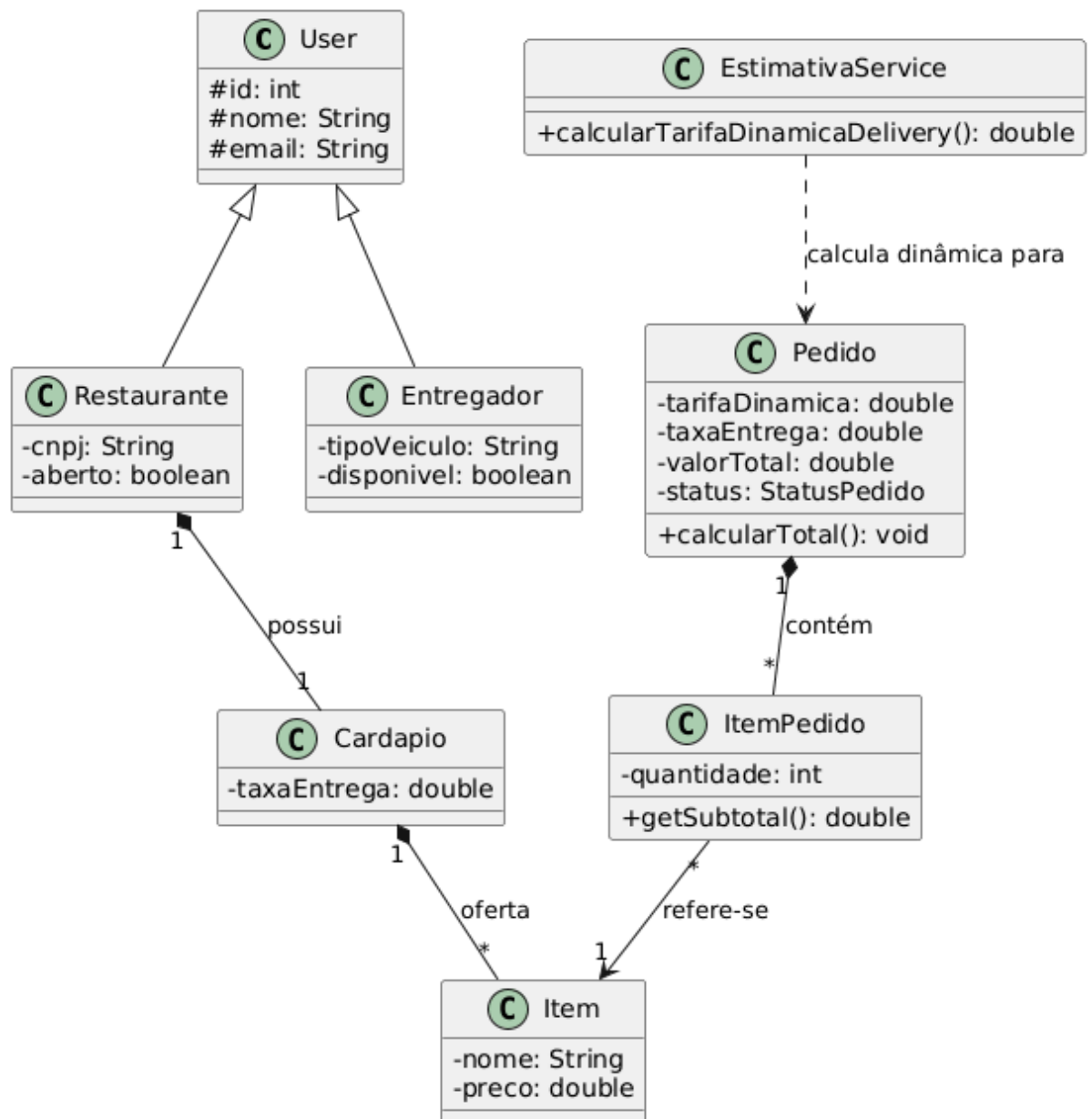
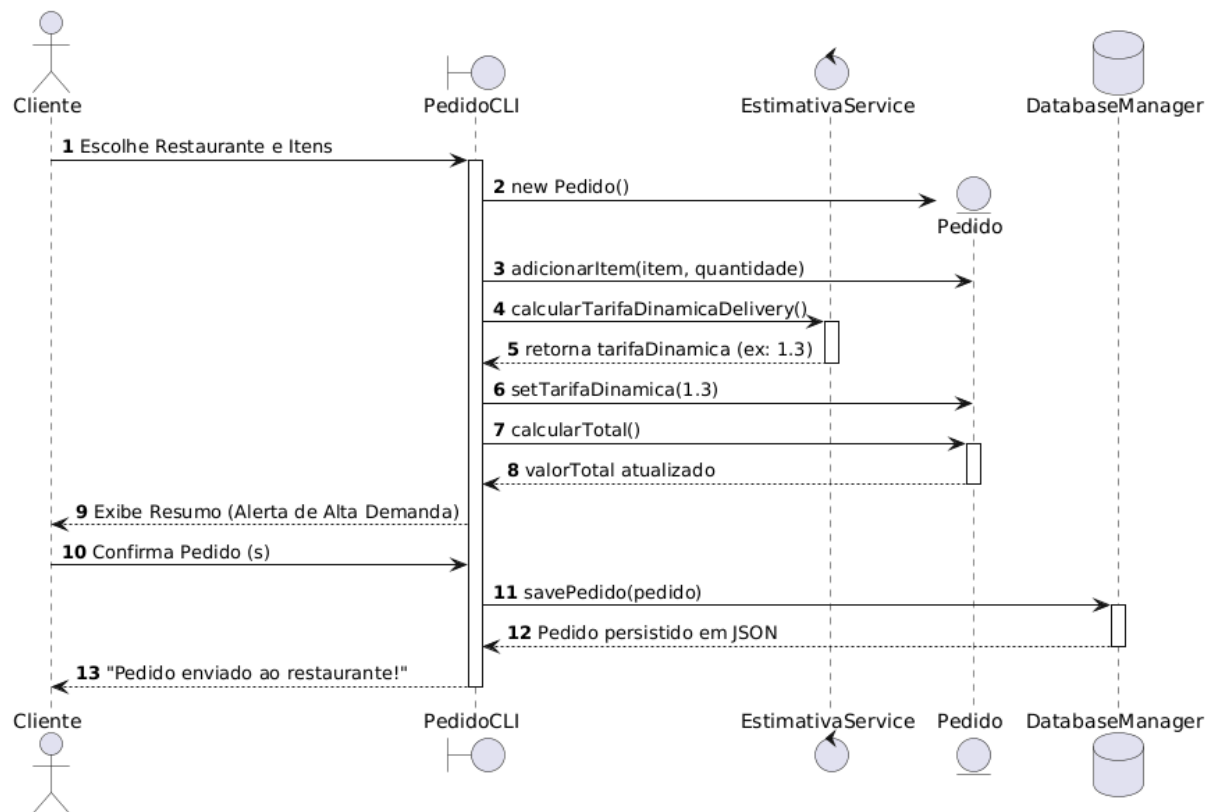


DIAGRAMA DE SEQUÊNCIA



FUNCIONALIDADES DESENVOLVIDAS

Nesta entrega, correspondente às Sprints 5 e 6, o sistema atingiu sua maturidade ao incorporar o fluxo de ponta a ponta do modelo Uber Eats. As principais funcionalidades entregues foram:

- **Gestão de Perfis de Delivery (RF01 e RF02):** Implementação de fluxos de cadastro exclusivos para Restaurantes (exigindo validação de CNPJ) e Entregadores (validação de CNH e restrição a motos ou bicicletas).
- **Gestão de Oferta e Cardápios (RF19 e RF20):** Restaurantes agora possuem autonomia para abrir e fechar o estabelecimento via sistema, além de cadastrar itens, descrições, preços e taxa base de entrega em seus cardápios digitais. O cliente só consegue visualizar estabelecimentos com o status ativo.
- **Ciclo de Vida do Pedido (RF21, RF24 e RF25):** Fluxo end-to-end de compras. O cliente seleciona itens, o sistema calcula o valor total, e o pedido é roteado para o painel do Restaurante. O estabelecimento tem autonomia para aceitar ou recusar a demanda e, posteriormente, atualizar o status para "Em Preparo".
- **Atribuição e Logística (RF22, RF26 e RF27):** Quando o pedido fica pronto, o sistema localiza entregadores com status de disponibilidade "Online". O entregador assume a corrida, seu status muda para indisponível e o cliente passa a acompanhar o trajeto da entrega.
- **Precificação Inteligente e Tarifa Dinâmica (RF14):** O fechamento do valor do pedido não é mais estático. O sistema simula condições adversas (como horários de pico ou chuvas) e, caso a alta demanda seja identificada, aplica automaticamente um multiplicador de tarifa dinâmica

sobre a taxa de entrega, notificando o cliente no recibo com total transparência antes da confirmação do pagamento.

CADASTRO DE RESTAURANTE

```
--- Cadastro de Perfil Restaurante ---  
CNPJ (14 dígitos): 12345678901234  
CNPJ válido!  
Razão Social / Nome do Restaurante: Restaurante do Manoel  
✓ Razão Social válida!  
Endereço completo: Centro  
✓ Endereço válido!  
  
🎉 Perfil de restaurante cadastrado com sucesso!
```

GESTÃO DO RESTAURANTE E CARDÁPIO

```
=== Menu Restaurante (Restaurante do Manoel) ===  
Status: ● ABERTO  
Avaliação: 0,0 ★ (0 avaliações)  
-----  
1 - Abrir / Fechar Restaurante  
2 - Gerenciar Cardápio  
3 - Ver pedidos recebidos  
9 - Voltar ao Menu Principal  
Escolha: 2  
  
=== Gerenciar Cardápio ===  
1 - Adicionar item  
2 - Remover item  
3 - Definir taxa de entrega  
4 - Definir tempo estimado  
5 - Listar itens  
9 - Voltar  
Escolha: 5  
1. Pizza - R$ 40,00
```

O PEDIDO, A TARIFA DINÂMICA E AGENDAMENTO

=== Escolha um Restaurante ===

1 - RestTest

2 - Restaurante do Manoel

Escolha: 2

=== Cardápio ===

1 - Pizza | R\$ 40,00

0 - Finalizar Pedido

Escolha item: 1

Quantidade: 2

Item adicionado!

=== Resumo do Pedido ===

Pizza x2 = R\$ 80,00

Taxa entrega base: R\$ 5,00

⚠️ TARIFA DINÂMICA ATIVA (1.3x) - Alta demanda na região!

Taxa entrega com dinâmica: R\$ 6,50

Total a Pagar: R\$ 86,50

=== TIPO DE ENTREGA ===

1 - Entrega Imediata

2 - Agendar Entrega

=== TIPO DE ENTREGA ===

1 - Entrega Imediata

2 - Agendar Entrega

Escolha uma opção: 2

Digite a data e hora (Ex: 25/12/2025 20:30): 17/03/2026 11:00

Pedido agendado para: 17/03/2026 11:00

Confirmar pedido? (s/n): s

📦 Pedido enviado ao restaurante!

FLUXO DE ACEITE DO RESTAURANTE

=== Menu Restaurante (Restaurante do Manoel) ===

Status:  ABERTO

Avaliação: 0,0  (0 avaliações)

1 - Abrir / Fechar Restaurante

2 - Gerenciar Cardápio

3 - Ver pedidos recebidos

9 - Voltar ao Menu Principal

Escolha: 3

=== PEDIDOS ===

1 - Pedido #14 | Status: AGUARDANDO_RESTAURANTE | Total: R\$ 86,50

Escolha o número do pedido (0 para voltar): 1

=== Gerenciar Pedido #14 ===

Status atual: AGUARDANDO_RESTAURANTE

1 - Confirmar (aceitar) pedido

2 - Rejeitar pedido

3 - Finalizar preparo

9 - Voltar

1

Aceitar pedido e iniciar preparo na cozinha? (s/n)

s

✓ Pedido movido para EM PREPARO.

=== Menu Restaurante (Restaurante do Manoel) ===

Status:  ABERTO

Avaliação: 0,0  (0 avaliações)

1 - Abrir / Fechar Restaurante

2 - Gerenciar Cardápio

3 - Ver pedidos recebidos

9 - Voltar ao Menu Principal

Escolha: 3

=== PEDIDOS ===

1 - Pedido #14 | Status: EM_PREPARO | Total: R\$ 86,50

ATRIBUIÇÃO E ENTREGA

```
=== Menu Entregador ===  
Veículo: MOTO  
Status: ● ONLINE  
Localização Atual: Centro  
Avaliação: 0,0 ★ (0 avaliações)  
-----  
1 - Alternar disponibilidade (Ficar Online/Offline)  
2 - Atualizar Localização  
3 - Ver pedidos disponíveis  
4 - Ver meu pedido atual  
9 - Voltar ao Menu Principal  
Escolha: 3  
  
=== PEDIDOS DISPONÍVEIS ===  
1 - Pedido #14 | Total: R$ 86,50  
2 - Pedido #15 | Total: R$ 46,50  
3 - Pedido #16 | Total: R$ 85,00  
  
Escolha o número do pedido (0 para voltar): 1  
  
--- Ação ---  
1 - Aceitar pedido  
2 - Recusar (Voltar à lista)  
Escolha: 1  
🚚 Pedido aceito com sucesso! Agora está EM_ENTREGA.
```

```
1 - Alternar disponibilidade (Ficar Online/Offline)
2 - Atualizar Localização
3 - Ver pedidos disponíveis
4 - Ver meu pedido atual
9 - Voltar ao Menu Principal
Escolha: 4
```

```
=== MEU PEDIDO ATUAL ===
Pedido #14 | Total: R$ 86,50
```

```
=== ROTA DA ENTREGA ===
📍 Entregador: Centro
↓
🍽️ Restaurante: Centro
↓
👤 Cliente: Nao definida
```

```
1 - Finalizar entrega
9 - Voltar
1
✅ Pedido entregue com sucesso!
```

```
=====
📄 RECIBO DE DELIVERY - UBER EATS
Pedido #14
Status Final: ENTREGUE
Total da Compra: R$ 86,50
Entregador Parceiro: Manoel barros
Data/Hora: 16/03/2026 11:38
=====
```

RELATÓRIO DE TESTES

ESCOPO

Esse relatório detalha os cenários e resultados das baterias de testes aplicadas ao Módulo de Delivery (Uber Eats) do sistema **UberPB**, desenvolvidas nas **sprints 5 e 6**. O escopo abrange a validação rigorosa das regras de negócio (Models e Services) através de testes unitários e de integração, bem como a homologação da interface de linha de comando (CLI) por meio de testes Ponta a Ponta (E2E).

O objetivo principal deste relatório é comprovar o atendimento integral aos Requisitos Funcionais estabelecidos (RF19 ao RF28, além das validações de base RF01 e RF02) e garantir que a aplicação está imune a falhas de transição de estado e cálculos financeiros. A suíte de testes automatizados atingiu a marca de **120 testes implementados**, com **100% de taxa de aprovação**.

ARQUITETURA DE TESTES

A arquitetura de testes foi projetada para isolar a camada de domínio da camada de persistência em arquivos JSON, garantindo testes rápidos e determinísticos.

- **Framework Base:** JUnit 5.
- **Ferramenta de Build e Execução:** Apache Maven.
- **Tipos de Testes Aplicados:**
 - **Testes Unitários:** Validação isolada de cálculos (ex: Tarifa Dinâmica, Subtotais de Carrinho) e validações de documentos (CNPJ/CNH).
 - **Testes de Integração:** Simulação de fluxo de dados entre diferentes atores (Ex: Restaurante → Sistema de Fila → Entregador).
 - **Testes Ponta a Ponta (E2E):** Execução manual simulada da interface CLI para atestar a experiência do usuário e persistência JSON real.

Testes Sprint 5

Foram criados testes para os requisitos da Sprint 5, e testes para completar a cobertura do sistema como um todo:

- **CardápioTest:** Testes para as funcionalidades e fluxos de adicionar e remover itens ao cardápio, e aos pedidos, além de verificar o valor da taxa de entrega.
- **PedidoTest:** Testes para fazer pedidos imediatos e agendados.
- **RestaurantTest:** Testes relacionados as avaliações dos restaurantes.
- **EntregadorServiceTest:** Testes pensando em relação aos requisitos de cadastro de entregador.
- **RestauranteServiceTest:** Testes pensando em relação aos requisitos de cadastro do restaurante.
- **TestCorridaRepositoryCompleto:** Testes de funcionalidades alteradas e adaptadas: findByCategoria e deleteById.

- **Testes de ponta a ponta:** Além disso, foram ajustados testes de ponta a ponta, com o objetivo de simular situações de um usuário real utilizando várias funcionalidades em sequência.

Testes Sprint 6

Foram criados testes para os requisitos da Sprint 6, e testes para completar a cobertura do sistema como um todo:

- **AvaliacaoETarifaDinamicaTest:** implementação de testes de avaliação, tarifas dinâmicas e cálculo/recálculo de média de avaliações.
- **PedidoAgendamentoECalculoTest:** implementação de testes de cálculo de itens no carrinho, e agendamentos de pedidos, e simulação de pedido agendado.
- **FluxoEntregadorIntegrationTest:** implementação de testes do aceitando pedido e ficando indisponível, entregador rejeita pedido e passa para o próximo entregador da fila, entregador finaliza a entrega e atualiza o status, e não permitir entregador pegar mais de um pedido de uma vez.

CENÁRIOS DE TESTE

Abaixo estão as principais cenários de testes desenvolvidas especificamente para o escopo do Delivery, garantindo a robustez das novas funcionalidades.

1. Cenário: PedidoAgendamentoECalculoTest (RF19 a RF23)

Focada na validação matemática do carrinho de compras e na máquina temporal do agendamento de pedidos.

- **Cenário Validado (Cálculo de Total):** Adição de múltiplos ItemPedido com diferentes quantidades, validando se a soma dos subtotais aliada à Taxa de Entrega do restaurante resulta no valor financeiro exato.
- **Cenário Validado (Agendamento RF23):** Bloqueio de agendamento retroativo. O sistema lança uma IllegalArgumentException caso o usuário tente injetar uma LocalDateTime que esteja no passado.
- **Transição Base:** Confirmação da mudança estrutural do status inicial CRIADO para AGUARDANDO_RESTAURANTE.

2. Cenário: AvaliacaoETarifaDinamicaTest (RF14 e RF28)

Responsável por validar a implementação da precificação inteligente e do sistema de feedback mútuo.

- **Cenário Validado (Tarifa Dinâmica - RF14):** Teste que injeta um multiplicador de "Horário de Pico" (ex: 1.5x) sobre a taxa de entrega base do pedido, garantindo que o getValorTotal() seja recalculado corretamente antes do fechamento.

- **Cenário Validado (Avaliação Mútua - RF28):** Injeção de notas para o Restaurante e para o Entregador. O teste valida a precisão matemática da média global flutuante (ex: recálculo exato de uma média 5.0 que cai para 3.0 ao receber uma nota 1.0).

3. Cenário: FluxoEntregadorIntegrationTest (RF24, RF25 e RF26)

Testa o ciclo de vida avançado do pedido na camada de serviços.

- **Cenário Validado (Bloqueio de Concorrência):** Ao aceitar um pedido (EM_ENTREGA), o entregador tem sua flag disponível setada para false, impedindo que ele receba novos chamados simultâneos.
- **Cenário Validado (Reatribuição Inteligente - RF25):** Simulação de um Entregador A rejeitando um pedido. O teste garante que o ID do Entregador A é desvinculado e o pedido é automaticamente repassado para o Entregador B da fila.

TESTES DE PONTA A PONTA (E2E) + FLUXO CLI

Abaixo, apresentamos os recortes da interface CLI evidenciando a fluidez do sistema em ambiente de produção local.

1. Visão do Restaurante (Recebimento e Preparo - RF26)

O Restaurante identifica o pedido na fila, aceita e, após o preparo, o sistema notifica o motoboy:

```
=== PEDIDOS NA FILA ===
1 - Pedido #101 | Status: AGUARDANDO_RESTAURANTE | Total: R$ 45,50
Escolha um pedido para atualizar o status: 1

Aceitar pedido e iniciar preparo na cozinha? (s/n)
> s
✅ Pedido #101 movido para EM PREPARO.

Comida pronta? Chamar entregador parceiro? (s/n)
> s
✅ Entregador Carlos foi notificado para retirar o pedido!
```

2. Visão do Entregador (Recusa, Aceite e Recibo - RF24, RF25 e RF14)

O motoboy recebe o alerta. Testa-se a funcionalidade de emitir o recibo ao finalizar:

 NOVO PEDIDO TOCANDO!

```

Pedido #101 | Ganho pela entrega: R$ 5,50
1 - ACEITAR CORRIDA | 2 - RECUSAR
Sua escolha: 1
✅ Pedido aceito! Pilote com segurança.

[...] (Após chegar ao destino)
Chegou no destino? Confirmar entrega ao cliente? (s/n)
> s
✅ Entrega concluída com sucesso! Parabéns.

=====
📄 RECIBO DE DELIVERY - UBER EATS
Pedido #101
Status Final: ENTREGUE
Total Pago pelo Cliente: R$ 45,50
Entregador Parceiro: Carlos
Data/Hora: 16/03/2026 13:51
=====

```

3. Visão do Cliente (Rastreo e Avaliação - RF27 e RF28)

O cliente visualiza o status convertido de Enum para uma linguagem amigável (RF27) e realiza a avaliação mútua (RF28).

```

=== MEUS PEDIDOS (UBER EATS) ===
Pedido #101 | Valor Total: R$ 45,50
Status Atual: ✅ Entregue! Bom apetite.
-----

=== AVALIAR PEDIDO UBER EATS ===
Escolha o número do pedido para avaliar: 1
Dê uma nota de 1 a 5 para a comida do restaurante UEPB Lanches: 5
✅ Restaurante avaliado com sucesso!
Dê uma nota de 1 a 5 para o motoboy Carlos: 5
✅ Entregador parceiro avaliado com sucesso!

```

CONCLUSÃO E RESULTADOS FINAIS

Os testes comprovam que o sistema **UberPB** cumpre com maestria os requisitos propostos para a Release 3.

A arquitetura orientada a serviços demonstrou flexibilidade para absorver a tarifa dinâmica do delivery sem corromper a precificação de mobilidade. A persistência baseada em JSON provou-se confiável como fonte da verdade, eliminando a necessidade de repositórios paralelos para a geração de históricos (RF28). O console de homologação registrou sucesso na execução da totalidade dos **120 casos de testes**, qualificando o software como pronto para produção.